PDS - Mid IMP PYQ Sol.

PYTHON FOR D.S. | BE05000231 | Sem 5 | GTU | Ch : 1-4

> **Compiled BY** : Manthan A. D. [in]

Chapter Reference (Mid Syllabus)

New Ch.	Covers
Ch. 1	Intro, Data Types, Operators, Decision Making, Loops, Functions
Ch. 2	File Handling, Exception Handling, Python-MySQL
Ch. 3	Data Science & Descriptive Statistics
Ch. 4	Exploring Python Libraries (SciPy, NumPy, Pandas)

M.IMP QUESTIONS

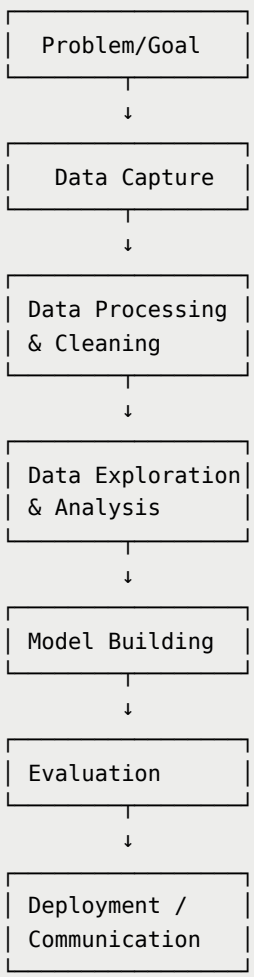
1. Explain Data Science Pipeline / Life Cycle, Stages and Phases

Chapter 3 | 7 Marks | Very Important

Answer

The **Data Science Life Cycle** is a systematic process used to convert raw data into useful information, insights, and decisions.

Main stages



Stages

1. **Problem Definition**
    - Identify the problem or business objective.
    - Define what needs to be predicted, analyzed, or discovered.
2. **Data Collection**
    - Collect required data from databases, files, APIs, websites, sensors, etc.

3. **Data Cleaning / Preparation**
    - Remove errors and duplicates.
    - Handle missing values.
    - Convert data into a useful format.
4. **Exploratory Data Analysis (EDA)**
    - Understand the data.
    - Find patterns, trends, relationships, and unusual values.
    - Use statistics and visualizations.
5. **Model Building**
    - Select suitable statistical or machine-learning techniques.
    - Train the model using prepared data.
6. **Evaluation**
    - Check whether the model/result solves the original problem.
    - Compare expected and obtained results.
7. **Deployment / Communication**
    - Use the final result in a real application.
    - Present findings through reports, dashboards, visualizations, etc.

Key Point

Data Science Life Cycle = Problem → Data → Cleaning → Analysis → Model → Evaluation → Deployment

Keywords: Problem Definition, Data Collection, Data Cleaning, EDA, Modeling, Evaluation, Deployment.

---

2. Pandas DataFrame Operations: shape, tail(), describe(), CSV, Slicing

Chapter 4 | 7 Marks | Theory + Code

What is a DataFrame?

A **Pandas DataFrame** is a two-dimensional data structure containing rows and columns.

```
import pandas as pd

data = {
    "Name": ["A", "B", "C"],
    "Marks": [80, 90, 75]
}

df = pd.DataFrame(data)

print(df)
```

Output:

```
  Name  Marks
0    A     80
1    B     90
2    C     75
```

Important DataFrame operations

1. `shape`

Returns the number of **rows and columns**.

```
print(df.shape)
```

Output:

```
(3, 2) # Means 3 rows , 2 columns
```

---

2. `tail()`

Displays the **last 5** rows.

```
print(df.tail()) # By Default ,Last 5 Rows
```

Output:

```
  Name  Marks
0    A     80
1    B     90
2    C     75
```

To display last 2 rows:

```
print(df.tail(2)) #Ignore - "0  A  80", From Upper Output
```

---

3. `describe()`

Provides **statistical summary of numerical columns**.

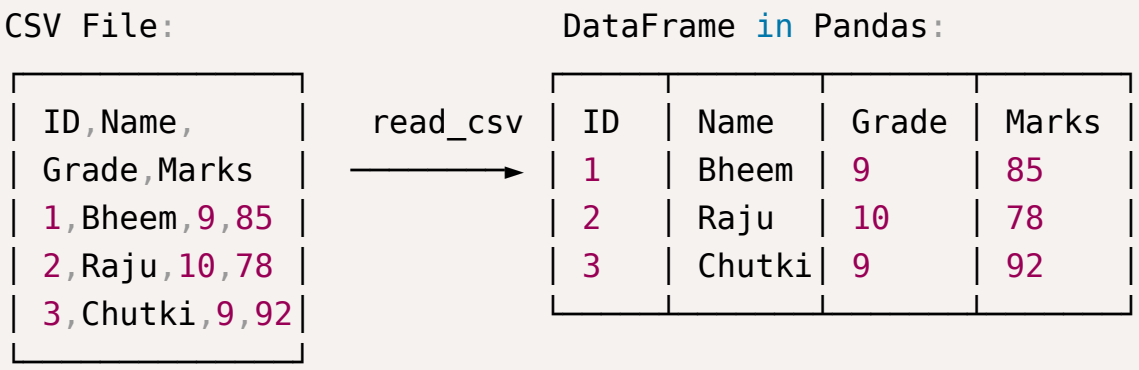
```
print(df.describe())  # Works only on numeric columns like Ma
rks
```

Output :

```
        Marks
count   3.000000
mean   81.666667
std     7.637626  # Standard deviation
min    75.000000
25%    77.500000
50%    80.000000
75%    87.500000
max    90.000000
```

---

4. Read CSV file




```python
# Let's consider a Random students.csv , With any radom value
df = pd.read_csv("students.csv") # read_csv() loads CSV data
=> DataFrame.
print(df)
```

Resulting DataFrame Output :

```
   ID    Name  Grade  Marks
0   1    Bheem      9     85
1   2     Raju     10     78
2   3   Chutki      9     92
```

---

5. Select a column

```python
print(df["Marks"]) # Selecting single column returns a Series
```

Output :

```
0    80
1    90
2    75
Name: Marks, dtype: int64
```

---

6. Select multiple columns

```
print(df[["Name", "Marks"]]) # Double brackets for multiple c
olumns
```

Output:

```
  Name  Marks
0    A     80
1    B     90
2    C     75
```

---

7. Select rows using `iloc`

```
print(df.iloc[0]) # first row as a Series
```

First row **Output :**

```
Name     A
Marks    80
Name: 0, dtype: object
```

First Two Rows :

```
print(df.iloc[0:2]) # first two rows(0 , 1) as a DataFrame
```

Output :

```
   Name  Marks
0    A     80
1    B     90
```

---

8. Select using `loc`

> **This selects a particular `row` and `column` using labels.**

```
print(df.loc[0, "Name"])
```

Output : `A`

Quick Revision

```
shape        → rows and columns
head()       → first rows**
tail()       → last rows
describe()   → statistical summary
read_csv()   → load CSV
iloc[]       → integer position
loc[]        → label-based selection
```

---

3. NumPy vs Pandas: Comparison and Characteristics

Chapter 4 | 4 Marks

NumPy

NumPy = Numerical Python

It is mainly used for:

- Numerical calculations
- Arrays
- Mathematical operations
- Multidimensional data

Example:

```
import numpy as np
```

```python
a = np.array([10, 20, 30])
print(a)

# Output:
# [10 20 30]
```

Pandas

Pandas is mainly used for:

- Data manipulation
- Data analysis
- Tabular data
- Data cleaning
- DataFrame and Series

Example:

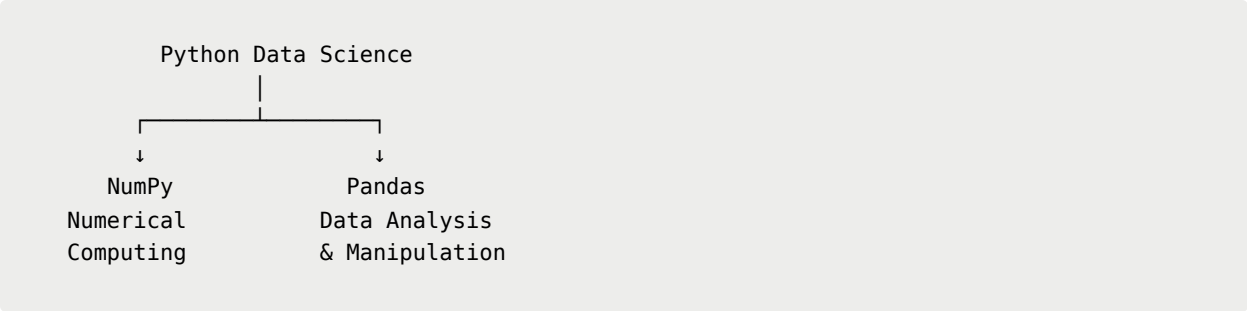
```python
import pandas as pd

df = pd.DataFrame({
    "Name": ["A", "B"],
    "Marks": [80, 90]
})
```

Difference

NumPy	Pandas
Mainly numerical computing	Mainly data analysis
Uses arrays	Uses Series and DataFrame
Best for mathematical operations	Best for tabular data
Mostly homogeneous data	Can handle different data types in columns
Fast array operations	Powerful indexing and data manipulation

Relationship



Remember:NumPy → Numbers/ArraysPandas → Tables/DataFrames

The supplied PYQ analysis identifies this as a **5× repeated topic**.

---

4. EDA: Definition and Tasks Involved

Chapter 3 | 7 Marks

Definition

Exploratory Data Analysis (EDA) is the process of examining and understanding a dataset before applying further analysis or modeling.

Its purpose is to discover:

- Patterns
- Trends
- Relationships
- Missing data
- Outliers
- Distribution of data

Major Tasks of EDA

1. Understand the dataset

Check:

- Number of rows
- Number of columns
- Data types
- Column names

Example:

```
df.shape
df.info()
df.head()
```

2. Handle missing values

```
df.isnull().sum()
```

This tells how many missing values are present.

3. Find duplicates

```
df.duplicated().sum()
```

4. Statistical analysis

```
df.describe()
```

Used to understand:

- Mean
- Standard deviation
- Minimum
- Maximum
- Quartiles

5. Detect outliers

Outliers are values that are unusually high or low compared with the rest of the data.

6. Study relationships

For example, study whether:

```
Study Hours  ↔  Exam Marks
```

are related.

7. Visualization

Graphs can be used to understand data easily.

Examples:

- Histogram
- Bar chart
- Scatter plot
- Box plot

EDA Flow

```
Raw Data
   ↓
Understand Data
   ↓
Clean Data
   ↓
Statistical Summary
   ↓
Find Patterns
   ↓
Detect Outliers
   ↓
Visualize Data
   ↓
Useful Insights
```

Key Point

EDA = Understand the data before making conclusions or models.

---

5. Python Data Structures: List vs Tuple vs Dictionary

Chapter 1 | 4 Marks

Python provides different structures for storing collections of data.

List

- Ordered
- Mutable
- Allows duplicate values
- Written using `[ ]`

```
a = [10, 20, 30]
a.append(40)
```

Tuple

- Ordered
- Immutable
- Allows duplicate values
- Written using `( )`

```
a = (10, 20, 30)
```

Dictionary

- Stores data in **key-value pairs**
- Mutable
- Written using `{ }`

```
student = {
    "name": "Manthan",
    "marks": 85
}
```

Comparison

Feature	List	Tuple	Dictionary
Syntax	`[ ]`	`( )`	`{key:value}`
Ordered	Yes	Yes	Key-value based
Mutable	Yes	No	Yes
Duplicate values	Yes	Yes	Keys must be unique
Main use	Collection	Fixed collection	Key-value data

Easy Memory Trick

```
LIST   → Can Change
TUPLE  → Cannot Change
DICT   → Key : Value
```

---

6. Skewness and Kurtosis

Chapter 3 | 7 Marks

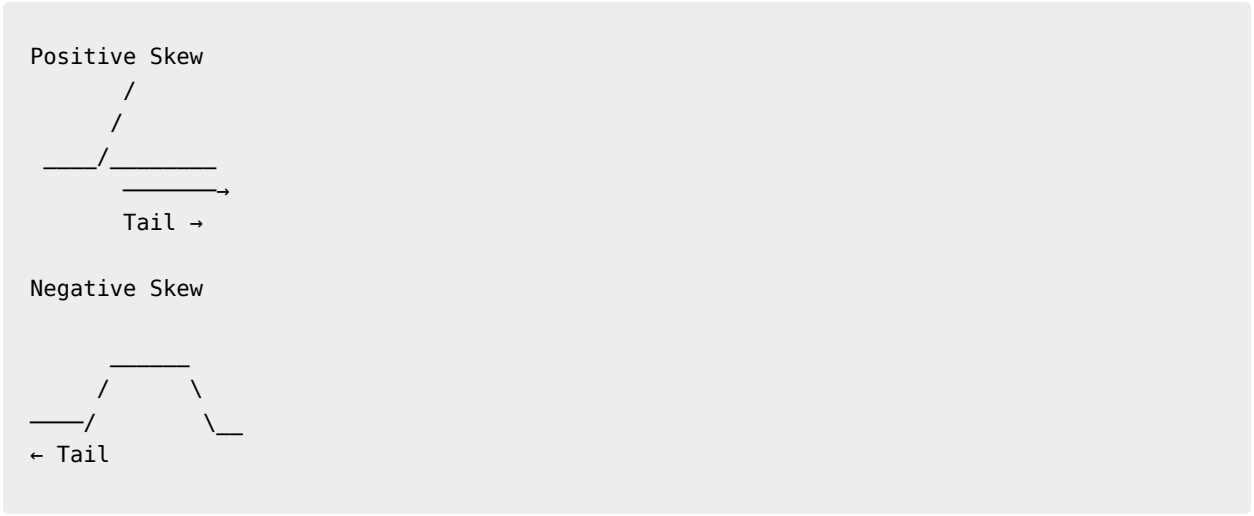
Skewness

Skewness measures the **asymmetry of a data distribution**.

A perfectly symmetrical distribution has approximately:

```
Skewness = 0
```

Types



Positive Skewness

- Tail extends toward the **right**.
- Mean is generally greater than median.

```
Mean > Median > Mode
```

Negative Skewness

- Tail extends toward the **left**.
- Mean is generally less than median.

```
Mean < Median < Mode
```

Zero Skewness

Distribution is approximately symmetrical.

Kurtosis

Kurtosis describes the shape or peakedness/tail behavior of a distribution.

Common classifications:

1. Mesokurtic

- Normal-like distribution.
- Moderate peak and tails.

2. Leptokurtic

- More peaked.
- Heavier tails.

3. Platykurtic

- Flatter distribution.
- Lighter tails.

```
Leptokurtic     Mesokurtic     Platykurtic

    /\            /\             _____
   /  \          /  \           /     \
  /    \        /    \         /       \
```

In Python

```python
from scipy.stats import skew, kurtosis

data = [10, 20, 30, 40, 50]

print(skew(data))
print(kurtosis(data))

# Output:
```

```
# 0.0
# -1.3
```

Remember:

```
Skewness → Asymmetry
Kurtosis → Shape / Tail characteristics
```

7. Covariance & Correlation, Definitions and Role in EDA

1. Covariance

- **Definition**: A statistical measure showing how two variables move together.
- **Range**: Negative infinity to positive infinity.
- **Meaning**: Positive covariance means they increase together. Negative means one goes up while the other goes down.
- **Limitation**: It only shows the direction of the movement, not the strength of the relationship.

2. Correlation

- **Definition**: The standardized version of covariance. It measures both the direction and the strength of the linear relationship between two variables.
- **Range**: Strictly between -1.0 and +1.0.
- **Meaning**: +1 is a perfect positive relationship. 0 means no relationship. -1 is a perfect negative relationship.
- **Best Metric**: Pearson correlation coefficient is the default standard in data science.

3. Role in EDA (Exploratory Data Analysis)

- **Feature Selection**: Helps identify which variables are strongly related to the target variable so you know what matters.

- **Spotting Multicollinearity**: Finds highly correlated independent variables. You drop redundant ones to keep models lightweight and accurate.
- **Missing Value Imputation**: If two variables are highly correlated, you can mathematically use one to predict and fill missing values in the other.
- **Pattern Recognition**: Feeds directly into correlation heatmaps, making it ridiculously easy to visually spot trends in a massive dataset.

IMP QUESTIONS

1. String Slicing and Name Extraction

Chapter 1 | 4 Marks

String Slicing

String slicing is used to extract a portion of a string.

Syntax

```
string[start:stop:step]
```

The `stop` index is not included.

Example:

```
name = "MAN THAN"
print(name[0:4])

# Output:
# MAN
```

A cleaner example:

```
name = "MANTHAN"
print(name[0:4])
```

```
# Output:
# MANT
```

Useful examples

```
s = "PYTHON"
                          # Output:
print(s[0:3])   # PYT
print(s[:3])    # PYT
print(s[3:])    # HON
print(s[-1])    # N
print(s[::-1])  # NOHTYP
```

Name extraction example

```
full_name = "Manthan D"

first_name = full_name.split()[0]
last_name = full_name.split()[1]

print(first_name)
print(last_name)
```

Output:

```
Manthan
D
```

---

2. Fibonacci Series / Factorial Program using Function or Recursion

Chapter 1 | 7 Marks

Fibonacci Series

Fibonacci sequence:

```
0 1 1 2 3 5 8 13 ...
```

Each term is obtained by adding the previous two terms.

Using loop

```python
n = 7
a = 0
b = 1

for i in range(n):
    print(a, end=" ")
    a, b = b, a + b

# Output:
# 0 1 1 2 3 5 8
```

Factorial using Function

Factorial of `n`:

```
n! = n × (n-1) × ... × 1
```

Example:

```
5! = 5 × 4 × 3 × 2 × 1
   = 120
```

Program:

```python
def factorial(n):
    result = 1

    for i in range(1, n + 1):
        result *= i

    return result

print(factorial(5))

# Output:
# 120
```

Factorial using Recursion

A recursive function calls itself.

```python
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))

# Output:
# 120
```

Recursion flow

```
factorial(5)
    ↓
5 × factorial(4)
    ↓
5 × 4 × factorial(3)
    ↓
5 × 4 × 3 × factorial(2)
```

```
    ↓
5 × 4 × 3 × 2 × factorial(1)
    ↓
120
```

Important: A recursive function must have a **base condition**, otherwise it keeps calling itself.

---

3. `range()` Function

Chapter 1 | 3 Marks

The `range()` function generates a sequence of numbers.

Syntax

```
range(start, stop, step)
```

`stop` is excluded.

1. `range(stop)`

```
for i in range(5):
    print(i)

# Output:
# 0 1 2 3 4
```

2. `range(start, stop)`

```
for i in range(2, 6):
    print(i)

# Output:
# 2 3 4 5
```

3. `range(start, stop, step)`

```
for i in range(2, 10, 2):
    print(i)

# Output:
# 2 4 6 8
```

4. Reverse sequence

```
for i in range(5, 0, -1):
    print(i)

# Output:
# 5 4 3 2 1
```

Key Point: `range()` is commonly used with `for` loops.

---

4. Reading Data from Text/CSV Files using Pandas

Chapter 2/4 | 4 Marks

Pandas provides functions for reading external data.

```
ID,Name,Grade,Marks
1,Bheem,9,85
2,Raju,10,78
3,Chutki,9,92
```

CSV file

```
import pandas as pd
```

```
df = pd.read_csv("students.csv")

print(df)
```

```
   ID    Name  Grade  Marks
0   1   Bheem      9     85
1   2    Raju     10     78
2   3  Chutki      9     92
```

Display first rows

```
print(df.head())
```

Display last rows

```
print(df.tail())
```

Check dimensions

```
print(df.shape)
# Output:# (3, 4)
```

Text file

For ordinary text files, Python file handling can be used:

```
file = open("data.txt", "r")
data = file.read()
print(data)
file.close()

# Output:
```

```
# Hello Python
# Welcome to Data Science
```

Better approach: `with`

```
with open("data.txt", "r") as file:
    data = file.read()
    print(data)
```

The `with` statement automatically handles closing the file.

Basic flow

```
CSV / Text File
      ↓
Read File
      ↓
DataFrame / Python Data
      ↓
Analyse
```

---

5. `groupby()` Function in Pandas

Chapter 4 | 3 Marks

`groupby()` is used to **group data based on one or more columns** and then perform operations on each group.

Example:

```
import pandas as pd

df = pd.DataFrame({
    "Department": ["CS", "CS", "IT", "IT"],
    "Marks": [80, 90, 70, 85]
})
```

```
result = df.groupby("Department")["Marks"].mean()

print(result)
```

Output:

```
Department
CS    85.0
IT    77.5
Name: Marks, dtype: float64
```

Conceptually:

```
CS → 80, 90 → Average = 85
IT → 70, 85 → Average = 77.5
```

Common operations

```
df.groupby("Department")["Marks"].sum()
df.groupby("Department")["Marks"].mean()
df.groupby("Department")["Marks"].max()
df.groupby("Department")["Marks"].min()
```

Key Point

`groupby()` **= Group similar data + perform an operation.**

---

6. Sorting and Shuffling DataFrame (Out_Of_Mid)

Chapter 4 | 4 Marks

```
Given:
90
```

```
70
80
```

Sorting

Sorting arranges data according to a column.

```
df.sort_values("Marks")
```

Ascending order:

```
df.sort_values("Marks", ascending=True)

# After ascending:
# 70
# 80
# 90
```

Descending order:

```
df.sort_values("Marks", ascending=False)

# After Descending:
# 90
# 80
# 70
```

Shuffling

Shuffling changes the order of rows randomly.

```
df = df.sample(frac=1)
```

`frac=1` means selecting 100% of the rows in random order.

Difference

```
Sorting
   ↓
Order based on a value

Shuffling
   ↓
Random order
```

---

7. Advantages / Role of Python in Data Science

Chapter 1/3 | 3 Marks

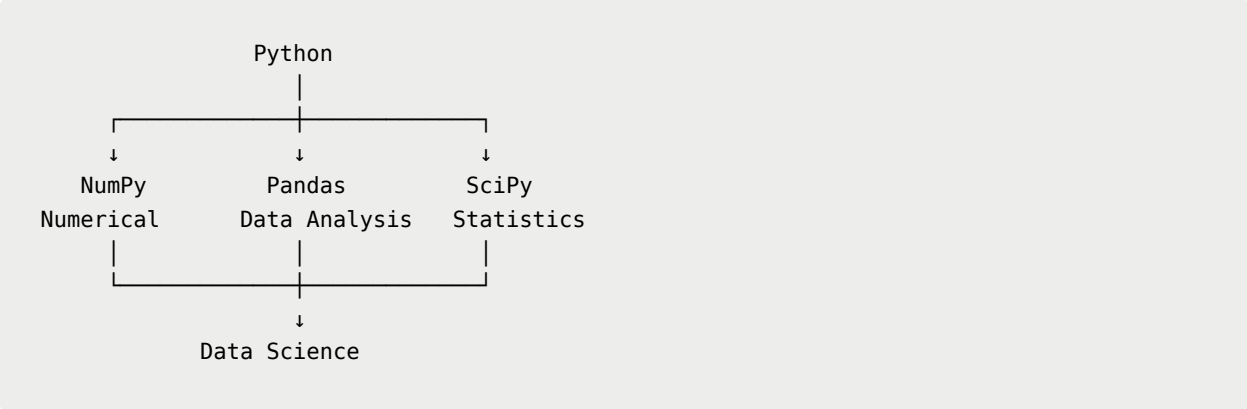
Python is widely used in data science because of its simple syntax and strong ecosystem of libraries.

Advantages

1. **Easy to learn**
   - Simple and readable syntax.
2. **Large number of libraries**
   - NumPy
   - Pandas
   - SciPy
   - Matplotlib
   - Scikit-learn
3. **Data manipulation**
   - Pandas makes cleaning and transforming data easier.
4. **Numerical computation**
   - NumPy provides efficient numerical operations.

5. **Statistical analysis**
    - SciPy and other libraries provide statistical functions.
6. **Visualization**
    - Python can create graphs and plots.
7. **Machine Learning**
    - Python provides libraries such as Scikit-learn.
8. **Large community**
    - Many learning resources and packages are available.

Simple representation



The official syllabus specifically emphasizes Python's role together with libraries such as Pandas and NumPy for data-science work.

📌 NEW SYLLABUS BLIND SPOTS

These are **not strongly supported by the supplied PYQs**. The analysis explicitly labels them as **"Zero PYQ Coverage"**, so study them after the M.IMP/IMP questions.

---

1. Exception Handling in Python

Chapter 2 | Predicted 4/7 Marks

Exception handling prevents a program from terminating unexpectedly when an error occurs.

Main keywords

```
try
except
else
finally
```

Basic example

```python
try:
    a = 10
    b = 0
    print(a / b)
except ZeroDivisionError:
    print("Cannot divide by zero")

# Output:
# Cannot divide by zero
```

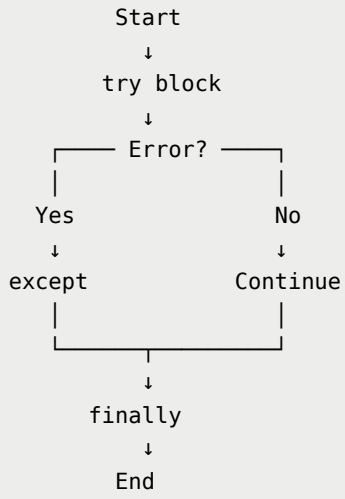
`finally`

`finally` executes whether an exception occurs or not.

```python
try:
    x = 10 / 2
except:
    print("Error")
finally:
    print("Execution completed")
```

```
# Output:
# Execution completed
```

Flow



Custom exception

```
try:
    age = int(input("Enter age: "))

    if age < 18:
        raise Exception("Age must be 18 or above")

except Exception as e:
    print(e)

# Output: (If Entered 16)
# Enter age: 16
# Age must be 18 or above
```

Remember

try → risky codeexcept → handle errorfinally → always execute

2. File Handling and Binary Files

Chapter 2 | Predicted 4 Marks

File handling is used to store and retrieve data from files.

Common modes

Mode	Meaning
`r`	Read
`w`	Write
`a`	Append
`rb`	Read binary
`wb`	Write binary

Text file

```
with open("data.txt", "w") as file:
    file.write("Hello Python")

with open("data.txt", "r") as file:
    data = file.read()
    print(data)

# Output:
# Hello Python
```

Binary file

Binary files store data in binary form.

```
with open("data.bin", "wb") as file:
    file.write(b"Hello")
```

```python
with open("data.bin", "rb") as file:
    data = file.read()
    print(data)

# Output:
# b'Hello'
```

Text vs Binary

```
Text File
   ↓
Human-readable characters

Binary File
   ↓
Bytes / binary data
```

---

3. Python-MySQL Basic Operations

Chapter 2 | Predicted 7 Marks

Python can communicate with MySQL using a suitable database connector.

Basic steps

```
Python Program
      ↓
Connect to MySQL
      ↓
Create Cursor
      ↓
Execute SQL Query
      ↓
Fetch / Modify Data
      ↓
Commit
      ↓
Close Connection
```

Example structure

```
import mysql.connector

mydb = mysql.connector.connect(
    host="localhost",
    user="root",
    password="password",
    database="college"
)

cursor = mydb.cursor()

cursor.execute("SELECT * FROM students")

result = cursor.fetchall()

for row in result:
    print(row)

mydb.close()
```

Important operations

Create connection

```
mysql.connector.connect(...)
```

Create cursor

```
cursor = mydb.cursor()
```

Execute query

```
cursor.execute("SELECT * FROM students")
```

Fetch records

```
cursor.fetchall()
```

Save changes

For `INSERT`, `UPDATE`, or `DELETE`:

```
mydb.commit()
```

Keywords

Connection → Cursor → Execute → Fetch/Modify → Commit → Close

---

4. SciPy `stats` Package

Chapter 4 | Predicted 3 Marks

SciPy is a Python library used for scientific and mathematical computing.

Its `stats` module provides many statistical functions.

```
from scipy import stats
```

Example: Mean

```
import numpy as np
from scipy import stats

data = np.array([10, 20, 30, 40])

print(stats.describe(data))
```

Output :

```
DescribeResult(
    nobs=4,
```

```
    minmax=(10, 40),
    mean=25.0,
    variance=166.66666666666666,
    skewness=0.0,
    kurtosis=-1.36
)
```

For Other statistical functions

```
print(stats.mode(data))
print(stats.skew(data))
print(stats.kurtosis(data))
```

The exact `stats.mode()` display can vary slightly by SciPy version, but for this data:

```
skew(data)     → 0.0
kurtosis(data) → -1.36
```

The source specifies the data `[10, 20, 30, 40]` and the `stats.describe()` program.

Uses

- Statistical analysis
- Probability distributions
- Descriptive statistics
- Hypothesis-related calculations
- Statistical tests

Remember

```
SciPy
  ↓
stats
  ↓
Statistical Functions
```

5. Bitwise, Membership and Identity Operators

Chapter 1 | Predicted 3 Marks

These operators are specifically listed in the official syllabus.

A. Bitwise Operators

Bitwise operators work on binary representations of numbers.

Operator	Meaning
`&`	AND
`|`	OR
`^`	XOR
`~`	NOT
`<<`	Left shift
`>>`	Right shift

Example:

```
a = 5
b = 3

print(a & b)
# Output:
# 1
```

Binary:

```
5 = 101
3 = 011
---------
    001 = 1
```

Therefore:

```
5 & 3 = 1
```

B. Membership Operators

Used to check whether a value exists inside a collection.

```
x in collection
x not in collection
```

Example:

```
numbers = [10, 20, 30]

print(20 in numbers)
print(50 not in numbers)

# Output:
# True
# True
```

C. Identity Operators

Used to check whether two variables refer to the **same object**.

```
is
is not
```

Example:

```
a = [1, 2]
b = a
```

```
print(a is b)

# Output:
# True
```

Do not confuse

```
==  → compares values
is  → compares object identity
```

This is a very common exam trap.

---

6. Data Measurement Scale

Chapter 3 | Predicted 4 Marks

A measurement scale describes how data is classified and what mathematical operations can meaningfully be performed on it.

There are **four major scales**.

1. Nominal

Used only for categories or names.

Examples:

- Gender
- Blood group
- Department
- Country

There is **no natural order**.

```
CS   IT   AI
```

---

2. Ordinal

Data has an order or ranking.

Examples:

```
Poor → Average → Good → Excellent
```

or:

```
1st → 2nd → 3rd
```

Difference between values is not necessarily equal.

---

3. Interval

Data has:

- Order
- Equal intervals

But **zero does not represent complete absence**.

Example:

```
Temperature in Celsius
```

The difference between 20°C and 30°C is the same as between 30°C and 40°C.

---

4. Ratio

Has:

- Order
- Equal intervals
- Meaningful zero

Examples:

- Height
- Weight
- Age
- Income
- Distance

Comparison

Scale	Order	Equal Difference	True Zero	Example
Nominal	No	No	No	Blood Group
Ordinal	Yes	No	No	Rank
Interval	Yes	Yes	No	Celsius
Ratio	Yes	Yes	Yes	Weight

Easy Trick

```
Nominal  → Name
Ordinal  → Order
Interval → Equal Interval
Ratio    → Real Zero
```

Ultra-Fast Mid Exam Revision Order

Based on the supplied PYQ analysis, I would revise in this exact order:

🔴 Tier 1: Do these first

1. **Data Science Life Cycle**
2. **Pandas DataFrame Operations**
3. **NumPy vs Pandas**
4. **EDA**

5. **List vs Tuple vs Dictionary**
6. **Skewness & Kurtosis**

The analysis gives these the M.IMP priority, with the first five having particularly strong recurrence.

🟡 Tier 2: Then these

1. String slicing
2. Fibonacci / Factorial / Recursion
3. `range()`
4. Reading CSV/Text files
5. `groupby()`
6. Sorting & Shuffling
7. Role of Python in Data Science

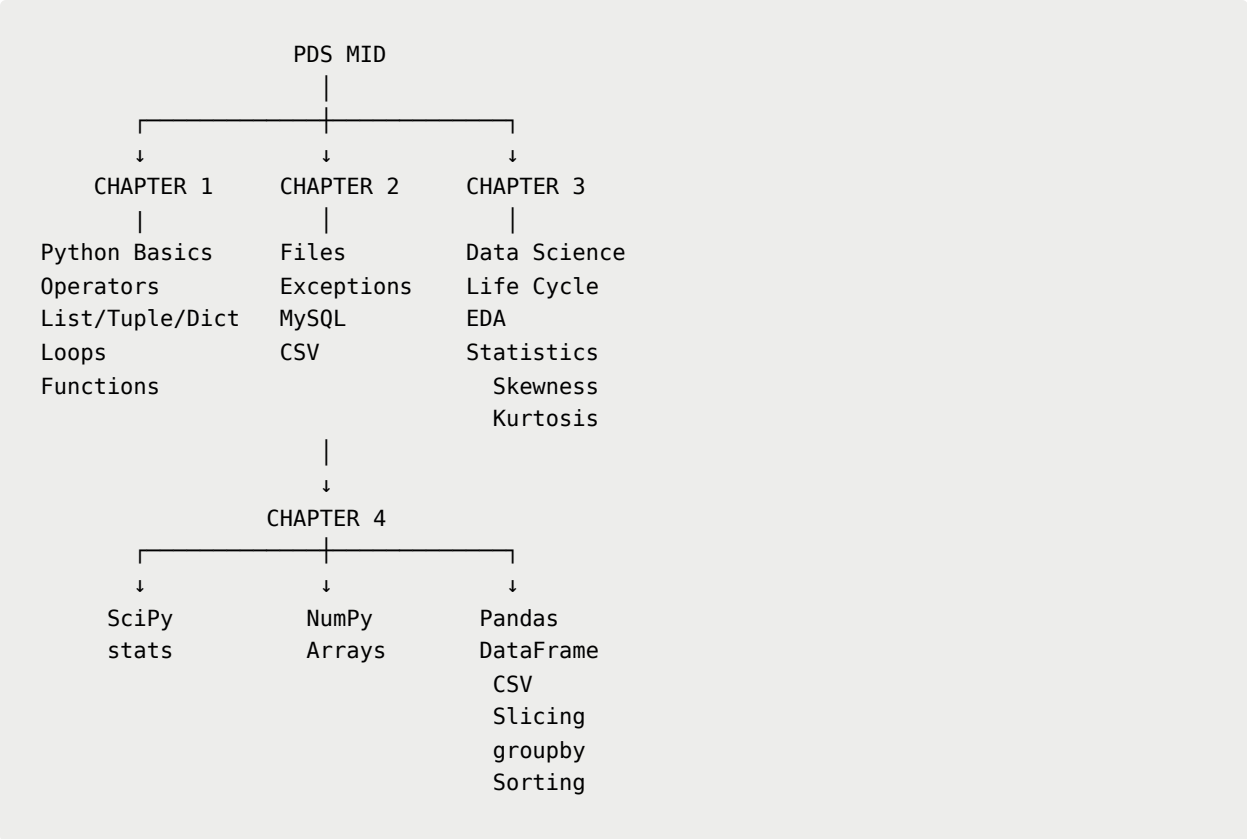
These are classified as IMP in the analysis.

📌 Tier 3: Blind Spots

1. Exception Handling
2. Binary File Handling
3. Python-MySQL
4. SciPy `stats`
5. Bitwise/Membership/Identity
6. Measurement Scales

These should be treated as **predictions rather than PYQ-backed questions**.

One-Page Memory Map



Final Exam Keywords

📎 **Python:** `list`, `tuple`, `dict`, slicing, loops, functions, recursion, operators
Data Science: `life cycle`, `EDA`, `analysis`, `data measurement`
Statistics: `skewness`, `kurtosis`, `mean`, `distribution`
NumPy: `array`, numerical computation
Pandas: `DataFrame`, `shape`, `tail()`, `describe()`, `read_csv()`, `iloc`, `loc`, `groupby()`, `sort_values()`
Ch. 2 blind spots: `try`, `except`, `finally`, `rb`, `wb`, `mysql.connector`, `cursor`, `execute()`, `commit()`

> **Compiled BY** : Manthan A. D. [in]